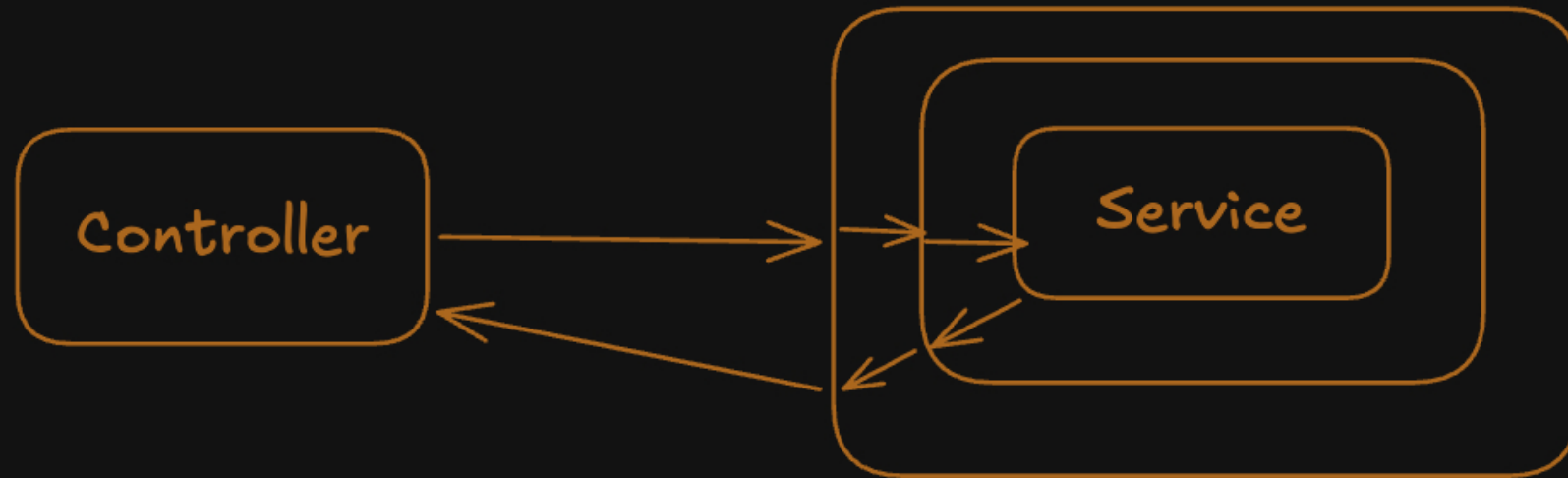


Introduction to AOP

Decorator Pattern



Proxy —————> Proxy Design pattern

AOP

-> Dependency

WHEN --> Before createStudent() execution

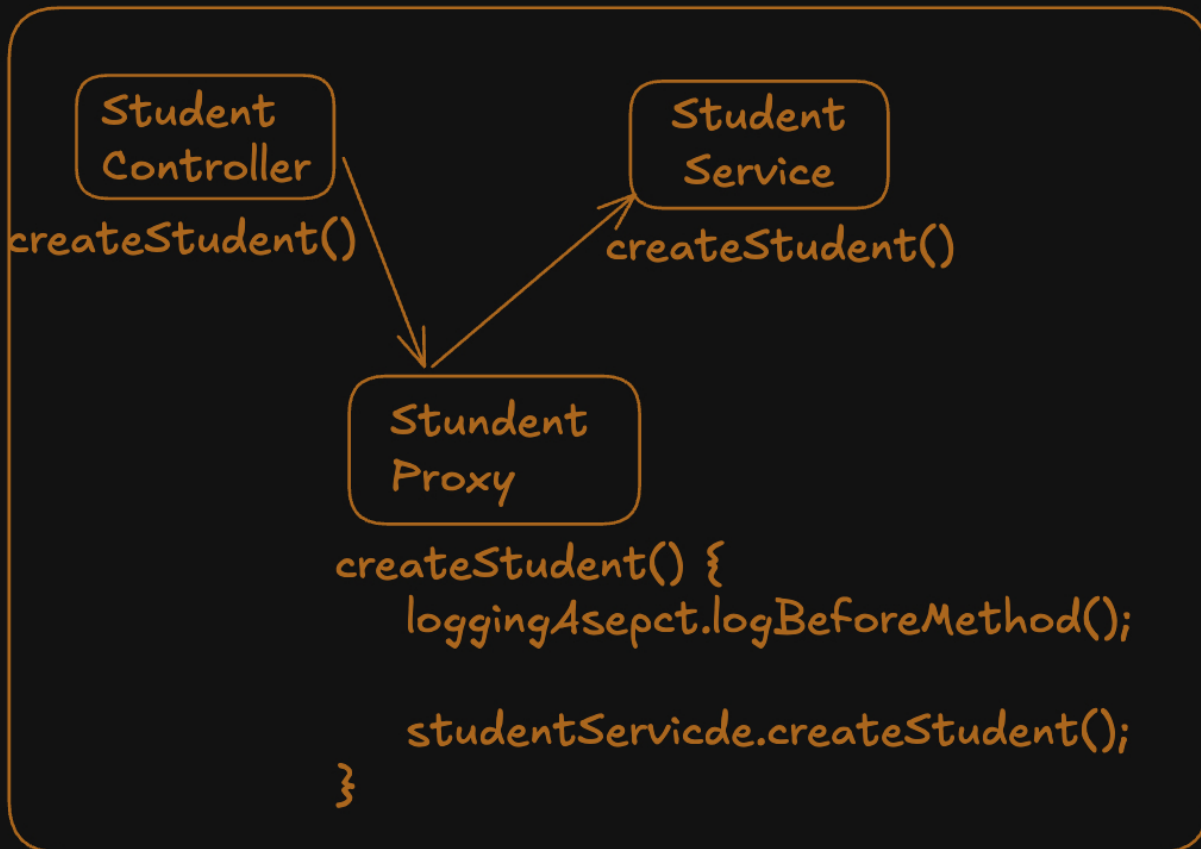
WHERE --> StudentService -> createStudent path

WHAT --> Logging logic

POINTCUT EXPRESSION

```
@Before("execution(String in.strikes.aopDemo.service.StudentService.createStudent())")  
public void logBeforeMethod() {  
    System.out.println("Student is going to be saved");  
}
```

ADVICE



```
@Before  
@After  
@AfterReturning  
@AfterThrowing  
@Around
```

```
beforeLogic();
```

→ @Before

```
try {
```

```
    Object result = targetMethod();
```

```
    afterReturningLogic(result);
```

→ @AfterReturning

```
    return result;
```

```
} catch (Throwable exception) {
```

```
    afterThrowingLogic(exception);
```

→ @AfterThrowing

```
    throw exception;
```

```
} finally {
```

```
    afterLogic();
```

→ @After

```
}
```